

SwiftDrop is a last-mile delivery platform helping logistics companies track parcels, manage delivery agents, and handle route assignments in real time. You're joining as a backend engineer and your first task is to build the core API that powers the entire operation — from the moment a parcel is registered to the moment it lands at a customer's door.

Core components:

- **Parcels** — register a parcel, update its status, get tracking history, list parcels by status or sender
 - **Delivery agents** — create an agent profile, set availability, assign a parcel to an agent, list an agent's active deliveries
 - **Delivery events** — log a delivery event (e.g. picked up, out for delivery, delivered, failed attempt), get the full event timeline for a parcel
 - **Reporting** — A summary per agent: total deliveries, success rate, average time from pickup to delivery
-

Technical requirements

1. **NestJS with Fastify adapter** — not Express
2. **Drizzle ORM with Postgres** — provide `schema.ts` and at least one migration using `drizzle-kit`
3. **Validation** — `class-validator` + `class-transformer` or Zod. Every incoming payload must be validated
4. **Error handling** — a global exception filter returning structured JSON. Cover 404s, invalid status transitions, and constraint violations
5. **Status machine logic** — parcel status must follow a valid transition path. Jumping from `registered` directly to `delivered` without going through `picked_up` and `out_for_delivery` should be rejected with a meaningful error
6. **API key guard** — a simple NestJS guard via env var. No OAuth needed
7. **Unit tests** — service layer tested with mocked repositories. At least cover the status transition logic thoroughly
8. **README** — setup, env vars, how to run migrations, how to run tests

Bonus (optional)

- A simple retry queue concept — failed delivery attempts can be re-queued (in-memory or a simple DB flag is fine, no need for actual Redis/BullMQ)
- **CI/CD with GitHub Actions** — a workflow that runs on every push to `main` or on pull requests. At minimum it should install dependencies, run the test suite, and fail the pipeline if any test fails. Extra credit for adding a lint step or a migration check (`drizzle-kit check`) as a separate job

Submission

Share a link to your public GitHub repository with all work committed.

A few things we expect to see in the repo before you submit:

- A proper `README.md` with setup instructions, env vars, how to run migrations, and how to run tests
- Commits that tell a story — avoid a single giant commit with everything in it. We're not looking for perfection, but a commit history that shows how you approached the problem is a good sign
- A `.env.example` file with all required environment variables listed (no real secrets)

Deadline: 5 days from the date you receive this assignment.

If you need an extension for any reason, reach out before the deadline — we're happy to accommodate genuine conflicts. Going silent and submitting late without notice is what counts against you, not asking for more time.

Once submitted, we'll review your repo internally and schedule a live session to walk through it together.